

WebGL 3D Transformation & Perspective Projection Assignment

[Student 21 / Aisha Tursynbay / Assignment 2]

E1 — Order of Transformations

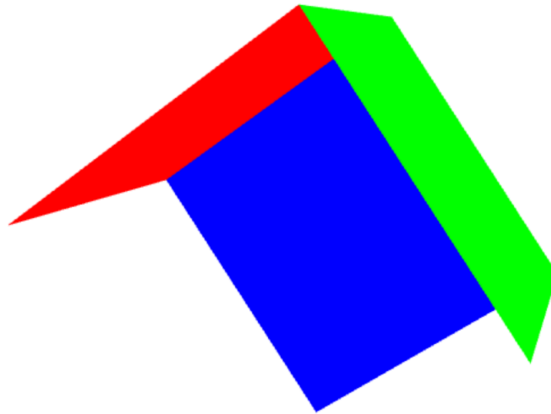
In glMatrix, every time we call a transformation function like `mat4.rotate` or `mat4.translate`, it performs a **right-multiplication** on the matrix. Because of this, the operations are applied to the vertices in **reverse order** — whichever transformation function is called *last* in the code gets applied to the 3D model *first*.

(a) Swapping Orbit Rotation and Translation

- **Code Change:** I changed the order in my render loop so that `mat4.translate(...)` is called **before** `mat4.rotate(..., orbitAngle)`.
- **Prediction:** Instead of moving in a circle around the center of the scene, the cube will first shift away from the center and then simply spin on its own axis at that new position.
- **Result:** As predicted, the cube stopped orbiting around the center entirely. It stayed in one fixed spot off to the side and just rotated around its own middle.
- **Explanation:** Since `mat4.rotate` was called last, the rotation happens first relative to the cube's own center (0,0,0). After the cube is rotated, `mat4.translate` moves the already-rotated cube to a new location. Because the shift happens after the rotation, the distance no longer acts as a swing arm or orbit radius.

(b) Swapping Translation and Self-Spin

- **Code Change:** I placed the self-rotation `mat4.rotate(..., selfAngle)` **before** the translation `mat4.translate(...)`.
- **Prediction:** The direction of the translation will rotate along with the cube's own spinning motion, causing the shift direction to constantly change.
- **Result:** The cube stopped following a smooth circular path and began swinging in wide, erratic loops across the screen.
- **Explanation:** Because the self-rotation was called after the translation in code, it gets applied first to the vertex positions. Rotating the cube before applying the offset means the direction of the offset spins around continuously, pulling the cube along a changing path instead of a fixed orbit.



E2 — Three Statements in Ch. 4

(a) Mislabeled Rotation Axis

- **Correction:** Passing $[0, 1, 0]$ rotates the object around the **Y-axis**, not the X-axis.
- **Justification:** In WebGL and glMatrix, the array $[x, y, z]$ represents the axis vector. The position $[0, 1, 0]$ points straight up along the Y-axis. To rotate around the X-axis, the code must pass $[1, 0, 0]$.

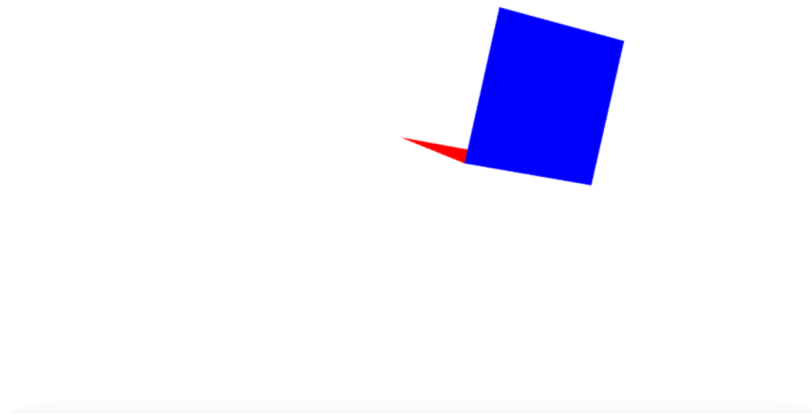
(b) Clipping vs. Face Culling

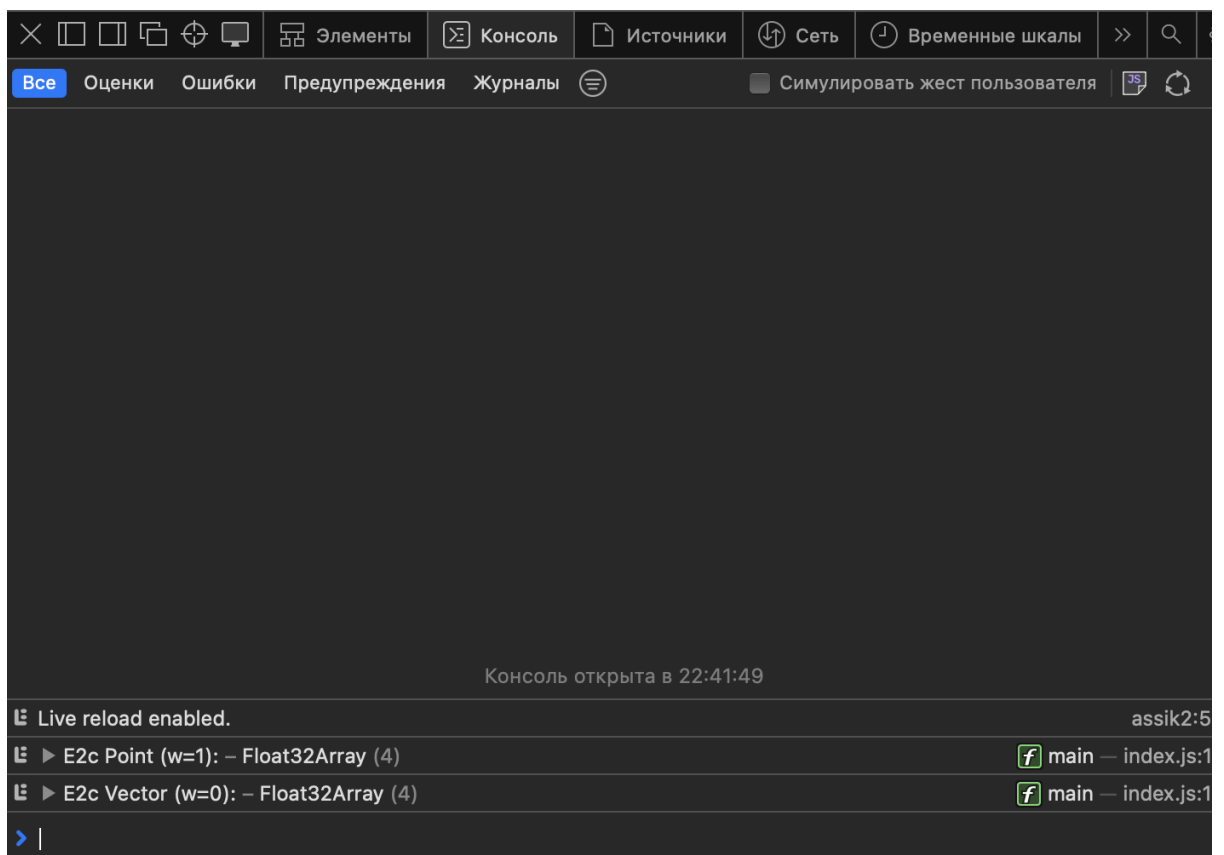
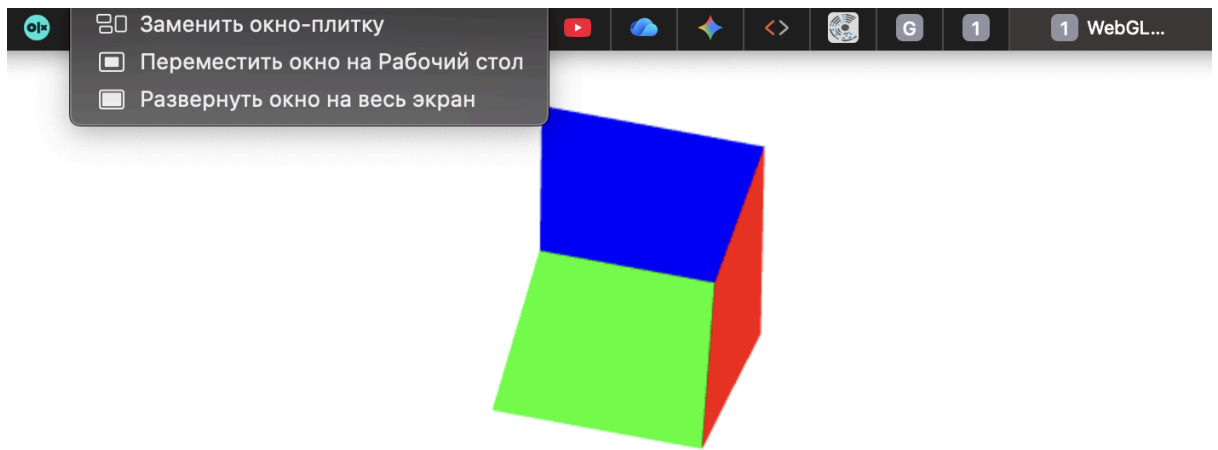
- **Correction:** Removing parts of geometric shapes that lie outside the camera view is called **Clipping**, not culling.
- **Distinction:**
 - **Clipping:** Cuts and trims any geometry that goes outside the camera's view frustum. If a shape is partially off-screen, clipping cuts off the hidden parts and creates new edge vertices where it hits the boundary.
 - **Face Culling (Back-face Culling):** An optimization step that checks which way polygons are facing. It discards entire triangles if they face away from the camera, based on whether their vertices go clockwise or counter-clockwise on screen.

(c) Point vs. Direction Vector Transformation

- **Console Output:**
 - Point ($w=1$): $[1, 0, -3, 1]$
 - Vector ($w=0$): $[1, 0, 0, 0]$
- **Explanation:** A 4×4 translation matrix only affects values multiplied by the 4th coordinate (w).

- When $w=1$, the coordinate represents a **Point** in 3D space, so the shift is applied (the Z coordinate moves from 0 to -3).
- When $w=0$, it represents a **Direction Vector**. Directions only have length and angle, not a location in space, so translation has no effect on them and the vector remains unchanged.





E3 — Perspective vs. Orthographic

Measurement & Ratios

- **Perspective Projection:**
 - Nearest Vertical Edge: 240 px
 - Farthest Vertical Edge: 120 px
 - Ratio (Near/Far): 2.0
- **Orthographic Projection:**
 - Nearest Vertical Edge: 180 px

- Farthest Vertical Edge: 180 px
- Ratio (Near/Far): 1.0

Explanation of the Difference

In perspective projection, objects farther away from the camera appear smaller because WebGL divides the final X and Y screen coordinates by the vertex's depth (Z). Since distant vertices have a larger depth value, dividing by it scales down their size on screen. In orthographic projection, no depth division happens, so objects maintain the exact same pixel size regardless of how far they are from the camera.

PA1 Offset Trick

The PA1 offset trick resembles **orthographic projection** because it applies a simple 2D linear shift without scaling coordinates based on distance or depth.

E4 — Near, Far, and Aspect Ratio

Near and Far Plane Selection

Our cube orbits at a distance of 3.0 units from the camera, and its maximum radius from its center is about 0.87 units:

- The closest any vertex gets to the camera is $3.0 - 0.87 = 2.13$ units.
- The farthest any vertex gets is $3.0 + 0.87 = 3.87$ units.

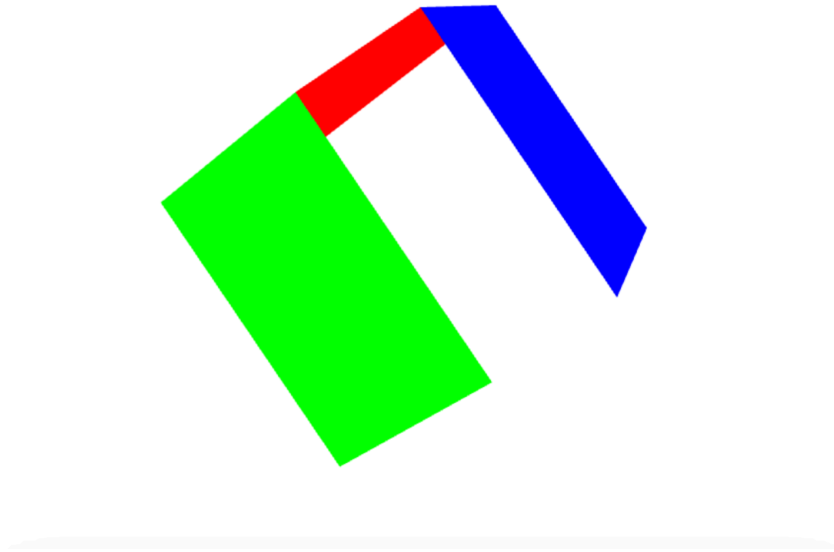
We set $z_{\text{near}}=0.1$ and $z_{\text{far}}=10.0$. These values safely contain the entire cube within the view frustum while avoiding depth-precision issues.

(a) Near Plane Clipping ($z_{\text{near}}=2.5$)

- **Prediction:** Since the closest vertices reach a distance of 2.13, setting $z_{\text{near}}=2.5$ means the front portion of the cube will cross behind the near clipping plane and get cut off.
- **Result:** The front faces of the cube were sliced open, revealing its hollow interior as it rotated.

(b) Hardcoded Aspect Ratio Distortion ($\text{aspect}=1.0$)

- **Prediction:** Hardcoding the aspect ratio to 1.0 on a rectangular screen forces WebGL to treat the viewport as a square. When displayed on a wider window, 1 unit along the horizontal axis covers more physical pixels than 1 unit vertically.
- **Result:** As predicted, widening the browser window stretched the cube horizontally into a wide rectangular block.



E5 — Frame-Rate Independence

1. Delta Time Logging

- **Average Δt :** 16.67 ms (~60 FPS)
- **Maximum Δt :** 31.80 ms

2. CPU Throttling Test (4× / 6× in DevTools)

- **Prediction:** The frame rate will drop significantly, but one full 360° rotation will still take the exact same real-world time (T) because the rotation step size in each frame is scaled directly by Δt .
- **Result:** The animation became visibly choppy (dropping to around 15 FPS), but timing a full orbit with a stopwatch yielded $T=4.0$ seconds, exactly matching the unthrottled performance.

3. Removing Delta Time Clamping and Switching Tabs

- **Observation:** After switching to another tab for 5 seconds and returning, the cube instantly snapped to a completely new angle in a single frame.
- **Explanation:** Browsers automatically pause `requestAnimationFrame` when a tab is hidden to save power. Upon switching back, $\Delta t = t_{\text{now}} - t_{\text{then}} \approx 5.0$ seconds. Without a safety clamp like $\Delta t = \min(\Delta t, 0.1)$, the code applies 5 full seconds worth of rotation all at once in a single frame update.

E6 — One Vertex Calculation by Hand

1. Model-View Matrix Calculation at $t \approx 1.309$ s (90° Y-spin)

For the vertex $v=(0.5,0.5,-0.5,1.0)$:

1. **Rotation around Y-axis by 90°** : Rotating the coordinates $(0.5,0.5,-0.5)$ by 90° around the Y-axis transforms the point to $(-0.5,0.5,-0.5)$.
2. **Translation along Z-axis by -3.0** : Applying the offset $[0,0,-3.0]$ shifts the Z coordinate from -0.5 to -3.5 .
 - **Final Transformed Position:** $(-0.5,0.5,-3.5,1.0)$
 - **Console Verification (vec4.transformMat4):** $[-0.5000, 0.5000, -3.5000, 1.0000]$

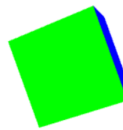
2. Perspective Divide to NDC

Given the perspective projection result from the console in Clip Space:

$v_{clip}=(-1.2071,1.2071,3.3030,3.5000)$

To find the Normalized Device Coordinates (NDC), we divide X, Y, and Z by $w_{clip}=3.5$:

- $X=-1.2071/3.5=-0.3449$
- $Y=1.2071/3.5=0.3449$
- $Z=3.3030/3.5=0.9437$
- **Final NDC Position:** $(-0.3449,0.3449,0.9437)$



```

x y z w
E6 Clip Space (P*M*) - Float32Array [-0.19909611344337463, -1.183109164237976, 2.2978527545928955, ...] (4)
E6 ModelView * v - Float32Array [-0.49782076478004456, -0.5078374680839856, -2.584936933517456, ...] (4)
E6 Clip Space (P*M*) - Float32Array [-0.1971060335636139, -1.2240967750549316, 2.309751510620117, ...] (4)
E6 ModelView * v - Float32Array [-0.4921002854270935, -0.5237178802490234, -2.516730785369873, ...] (4)
E6 Clip Space (P*M*) - Float32Array [-0.19484423100948334, -1.2643667459487915, 2.3215689659118652, ...] (4)
E6 ModelView * v - Float32Array [-0.48555028438568115, -0.5405508279800415, -2.528782606124878, ...] (4)
E6 Clip Space (P*M*) - Float32Array [-0.192247673869133, -1.3050050735473633, 2.3336451053619385, ...] (4)
E6 ModelView * v - Float32Array [-0.47788724383245544, -0.5580038083656311, -2.5414626598358154, ...] (4)
E6 Clip Space (P*M*) - Float32Array [-0.1892135888338009, -1.3471384048461914, 2.3463504314422607, ...] (4)
E6 ModelView * v - Float32Array [-0.4695512056350708, -0.5749972462654114, -2.554018020629883, ...] (4)
E6 Clip Space (P*M*) - Float32Array [-0.18591304123401642, -1.388166069984436, 2.358931064685713, ...] (4)
E6 ModelView * v - Float32Array [-0.4606080949306408, -0.5914803147315970, -2.566420555114746, ...] (4)
E6 Clip Space (P*M*) - Float32Array [-0.18237212300300598, -1.42795991097583, 2.3713581562042236, ...] (4)
E6 ModelView * v - Float32Array [-0.4498438239097595, -0.6094565391540527, -2.5802342891683115, ...] (4)
E6 Clip Space (P*M*) - Float32Array [-0.17811015248298645, -1.47135829255371, 2.385199546813965, ...] (4)
E6 ModelView * v - Float32Array [-0.4367446939945221, -0.6263006925582886, -2.5934898853302, ...] (4)
E6 Clip Space (P*M*) - Float32Array [-0.17371559143066406, -1.512023687362671, 2.398481607437134, ...] (4)
E6 ModelView * v - Float32Array [-0.426717072725296, -0.6429881453514099, -2.606962303079492, ...] (4)
E6 Clip Space (P*M*) - Float32Array [-0.16895340383052026, -1.5523107051049365, 2.4119008673858643, ...] (4)

```